TECHNICAL ASSIGNMENT REPORT

Computer Vision Pipeline Development: Object Segmentation, Quantitative Evaluation & Real-Time Tracking

Candidate: Vivin Viju | **Environment:** Google Colab (T4 GPU) & local Windows (Python 3.13 / OpenCV)

Dataset: Roboflow Egg Dataset (egg.v1i.folder), with the **train** split used for development and the **valid** split (22 images) used for evaluation

This report covers the whole process of building a classical computer vision pipeline that segments and tracks eggs in uncontrolled, messy scenes. I've included every experiment I tried, the bugs I hit and the numbers that surprised me, not just the final working code. The work ran in two places: foundation-model inference on Google Colab to build a ground-truth set, and low-latency classical OpenCV code on my own Windows machine.

> **At a glance.** SAM (run on Colab) provided the ground-truth masks. Of three classical methods, fixed HSV thresholding and Otsu thresholding both failed on the wider dataset, while Canny edges + dilation + largest-contour filtering worked best. The first IoU runs scored 13.53% and then 11.02%; tracing those numbers showed problems in the ground-truth masks and in my own pipeline, both explained in Task 3. The pipeline locks onto a single egg well in real time, but merges neighbouring eggs into one box, which is its main known weakness.

1. Introduction & System Architecture

I'm an M.Tech graduate in Robotics and Automation with a mechanical engineering background, so I tend to treat a vision problem as one part of a working system rather than a stand-alone algorithm. Any method has to cope with real conditions: lighting that shifts from image to image, shadows, and surfaces with very little contrast. To handle that, I split the pipeline across two environments:

- **Cloud compute (Google Colab):** a zero-shot foundation model (SAM) generates the ground-truth masks used for evaluation, so no manual annotation was needed.
- **Local execution (Windows / Python / OpenCV):** classical feature-extraction and morphology methods that I developed, tuned and evaluated on my own machine, with real-time use on edge devices in mind.

The sections below follow the order I actually worked in: ground truth (Task 1), the classical pipeline and its failed attempts (Task 2), evaluation and debugging (Task 3), and real-time tracking (Task 4).

2. Task 1: AI Ground Truth Generation (Segment Anything Model)

To evaluate my OpenCV pipeline without annotating images by hand, I used Meta's Segment Anything Model (SAM) through the Ultralytics library on Google Colab. The Roboflow dataset (`egg.v1i.folder`) was uploaded with its **train** and **valid** splits kept apart.

I wrote a Colab script to process the evaluation split. To stop old results from mixing with new ones, it first deletes the previous output folder with `shutil.rmtree('/content/runs')`, then runs inference on `/content/dataset/valid/egg`, and finally zips the predictions and downloads them.

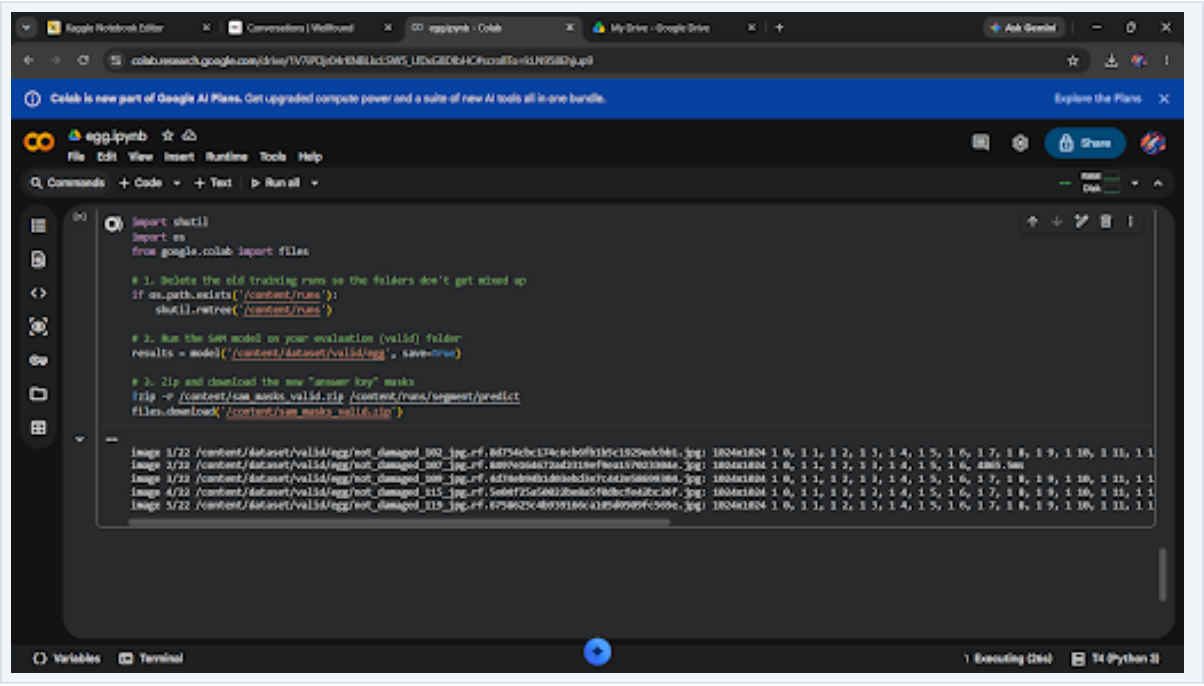


Figure 1: *Running Ultralytics SAM inference in Google Colab on the 22 validation-split images and archiving the output predictions.*

SAM was run without any prompt, so it tried to segment everything it could find in each picture: the egg, but also the hand, bowls, cloth and background. I also ran it on the training split to look at the results (Figure 2); the validation outputs are in Figure 3.

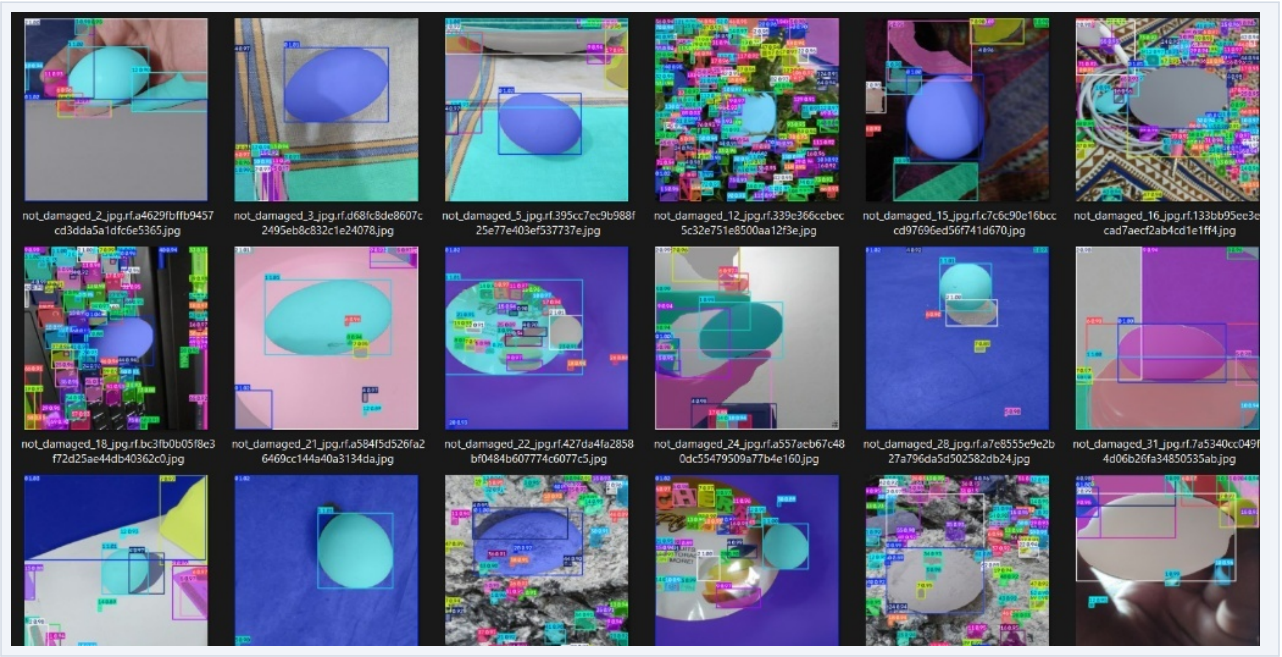


Figure 2: *SAM outputs on the training split. Each region is coloured as a separate segment (multi-class overlay).*

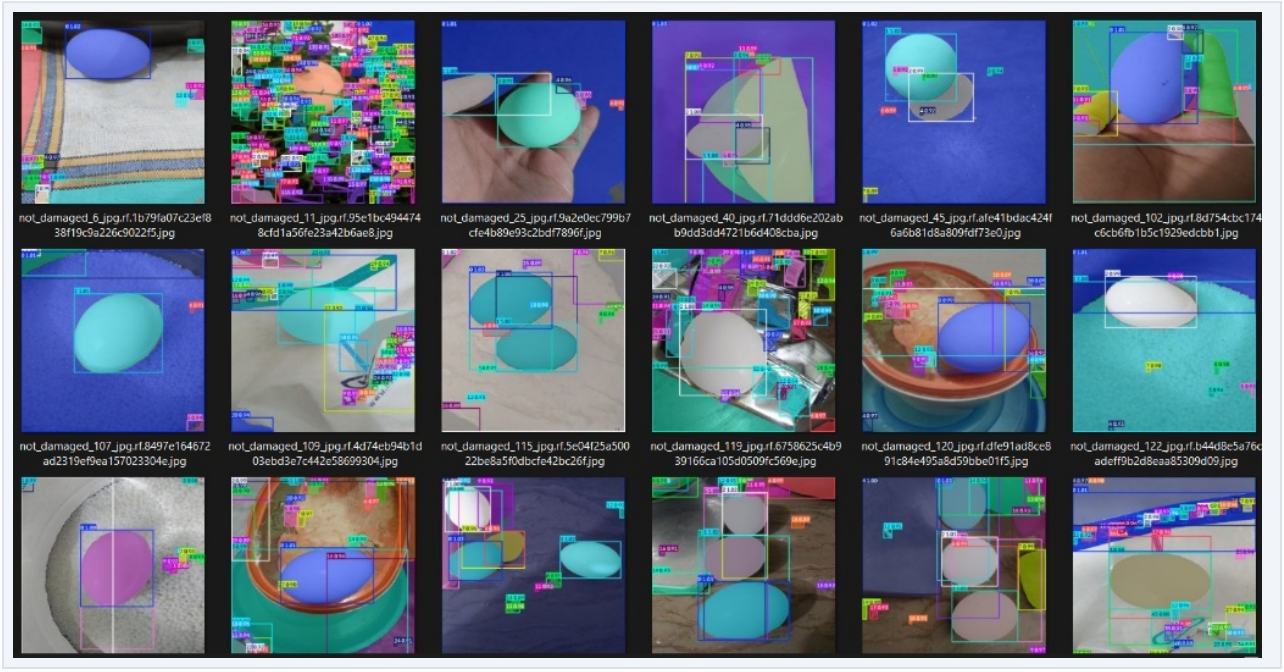


Figure 3: *SAM outputs on the validation split, with the same multi-segment overlays.*

Initial output formatting issue. With `save=True`, the SAM script saved visualization images (the original picture with a coloured, semi-transparent mask on top) instead of raw binary masks. This didn't cause trouble until the evaluation stage, where grading needed true binary masks. I fixed it with a second script that pulls out the raw polygon coordinates (`result.masks.xy`) and redraws them as pure binary masks. The details are in Task 3.

3. Task 2: Classical OpenCV Pipeline & Iterative Trials

With the dataset ready, I moved to my local Windows machine to build a segmentation pipeline in OpenCV, with the goal of separating the egg from both the background and the hand holding it, without any neural network. I tried three strategies in turn.

Trial 1: Fixed HSV Colour-Space Thresholding

My first guess was that the eggshell colour could be picked out in HSV space. I built a small interactive GUI with `cv2.createTrackbar` so I could tune the lower and upper bounds live while watching the mask.

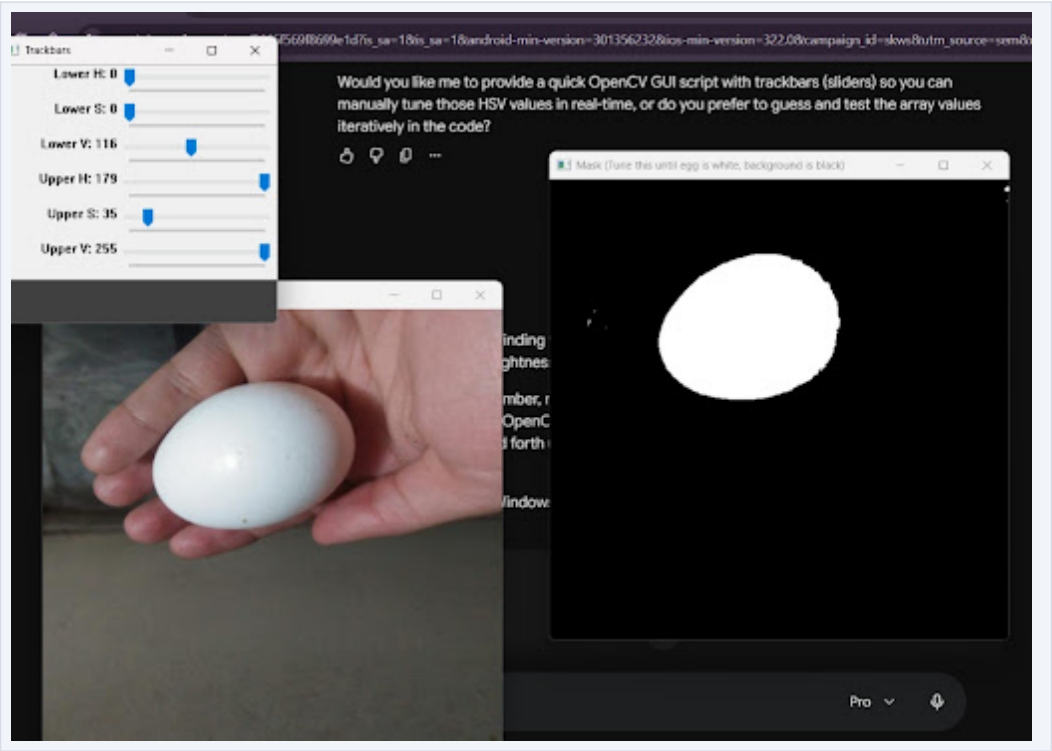


Figure 4: *Real-time HSV trackbar tuning. The best bounds for this one image were Lower: [0, 0, 116] and Upper: [179, 35, 255].*

Result: failed. The tuned values worked well on the single test image, but a hard-coded threshold fell apart once I looped it over the rest of the dataset. Changes in ambient lighting, shadows from the fingers and different shell tints made the mask either lose large parts of the egg or pull in a lot of background.

Trial 2: Otsu's Automatic Thresholding

To deal with changing light, I switched to a threshold that adapts to each image: Otsu's method on a grayscale, Gaussian-blurred frame. Otsu picks the cut between foreground and background intensities automatically.

Result: failed. Shadows on the egg and the skin tone of the hand have almost the same grayscale brightness, so there was no clean two-peak histogram to split. The fingers kept getting merged into the egg mask.

Trial 3: Canny Edge Detection + Dilation + Contour Filtering (best of the three)

Since colour and brightness were both unreliable, I stopped looking at pixel intensity and looked at shape instead. The pipeline has four stages:

1. **Smoothing:** convert to grayscale and apply a (9, 9) Gaussian blur to suppress fine skin texture.
2. **Canny edge detection:** find sharp gradients using thresholds 30 and 100.
3. **Morphological dilation:** a 5×5 kernel for 2 iterations (`cv2.dilate`) to close the small gaps in the edge outline. The raw Canny output often had tiny breaks that kept the egg's outline from closing.
4. **Contour isolation:** extract external contours (`RETR_EXTERNAL`) and keep only the one with the largest area, using `max(contours, key=cv2.contourArea)`, then fill it to get the mask.

Edge detection always picks up some background texture, which is why the last step matters. Keeping only the largest contour throws away the small specks and leaves one clean, filled object, as Figure 5 shows.

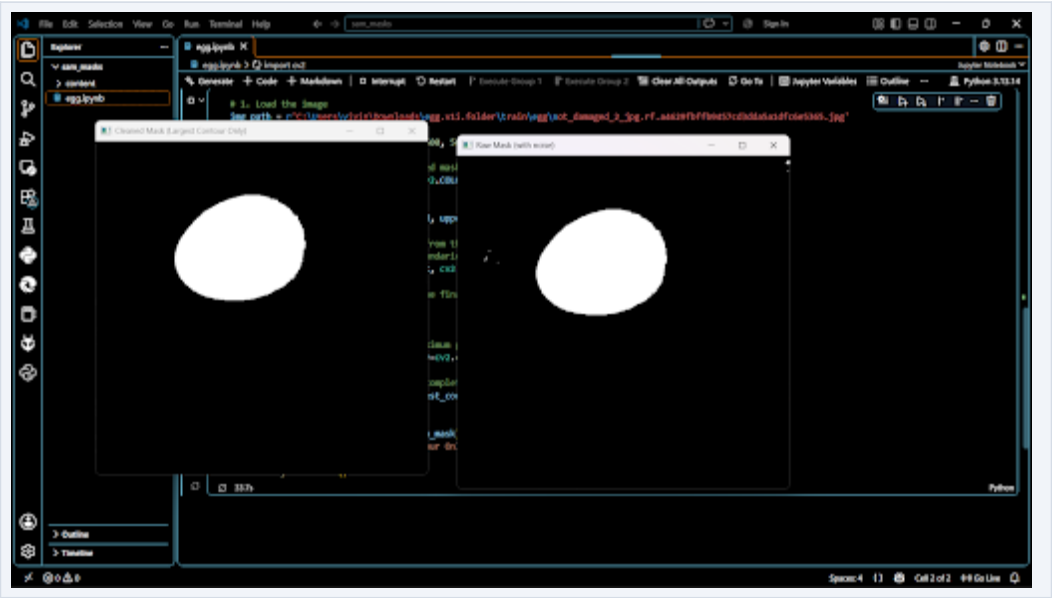


Figure 5: *Left window: cleaned mask keeping only the largest contour. Right window: raw mask, with small noise specks still visible.*

Result: best of the three. This approach doesn't depend on pixel colour or brightness, so changes in lighting and skin tone bothered it far less than they bothered the first two methods. Judged by eye on the development split, it handled single-egg images well. Its limits on multi-egg and cluttered images are covered in Tasks 3 and 4.

Summary of the three trials

Trial	Method	Outcome	Why
1	Fixed HSV thresholds (trackbars)	Failed	Thresholds tuned on one image did not carry over to other lighting, shadows or shell tints.
2	Otsu on blurred grayscale	Failed	Egg shadows and hand skin have similar brightness, so no clean split; hand merged into the mask.
3	Canny + dilation + largest contour	Best of the three	Uses shape/edges instead of colour; largest-contour step removes background noise.

4. Task 3: Quantitative Evaluation & Debugging Discrepancies

To grade the Canny pipeline with numbers, I wrote a script that computes Intersection over Union (IoU) between my classical masks and the SAM ground-truth masks over the 22 images in the validation split. Any image scoring below 0.70 is saved to a `bad_samples` folder so I can inspect it by eye.

Debugging Issue 1: Silent Execution & Path Syntax Errors

My first local run finished in 0.0 s and printed nothing. Two things were wrong: nested quotes inside raw strings (`r'"..."'`) and a missing `*.jpg` wildcard in `glob.glob()`. I cleaned up the path strings and added heartbeat print statements so I could see the script was actually running.

Debugging Issue 2: The 13.53% IoU (RGB Overlay Bug)

With the paths fixed, the script reported an average IoU of **13.53%**. The `bad_samples` folder showed why: the SAM masks were solid white squares. Ultralytics SAM with `save=True` had written RGB visualization overlays, not binary tensors, so thresholding them turned the entire canvas white. The script was comparing a well-shaped egg mask (roughly 13% of the image) against a mask covering 100% of the image, so the union area was huge and the IoU collapsed.

Debugging Issue 3: The 11.02% Result with Unprompted SAM

I went back to Colab and wrote a loop that reads the polygon coordinates from `result.masks.xy` and draws them as pure binary masks (`pure_masks.zip`). Re-running the local evaluation against these corrected masks gave an average IoU of **11.02%**, which was still very low.

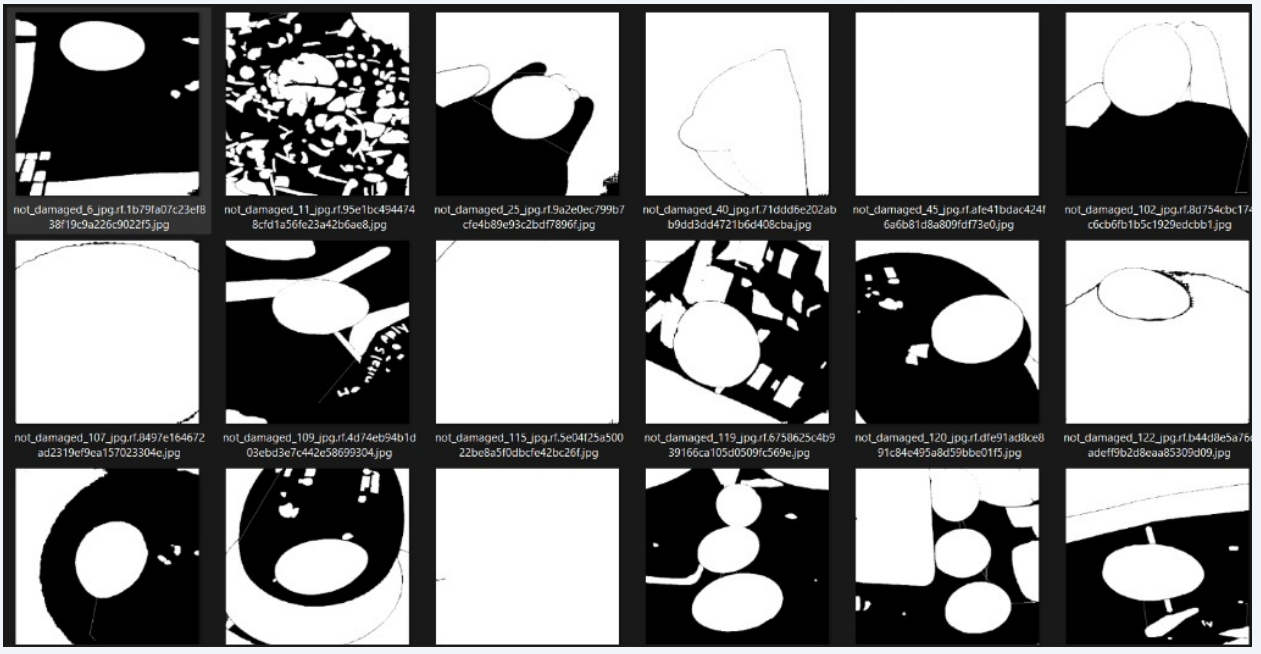


Figure 6: *Pure binary ground-truth masks rebuilt from the SAM polygon coordinates (validation split). Several masks are almost completely white.*

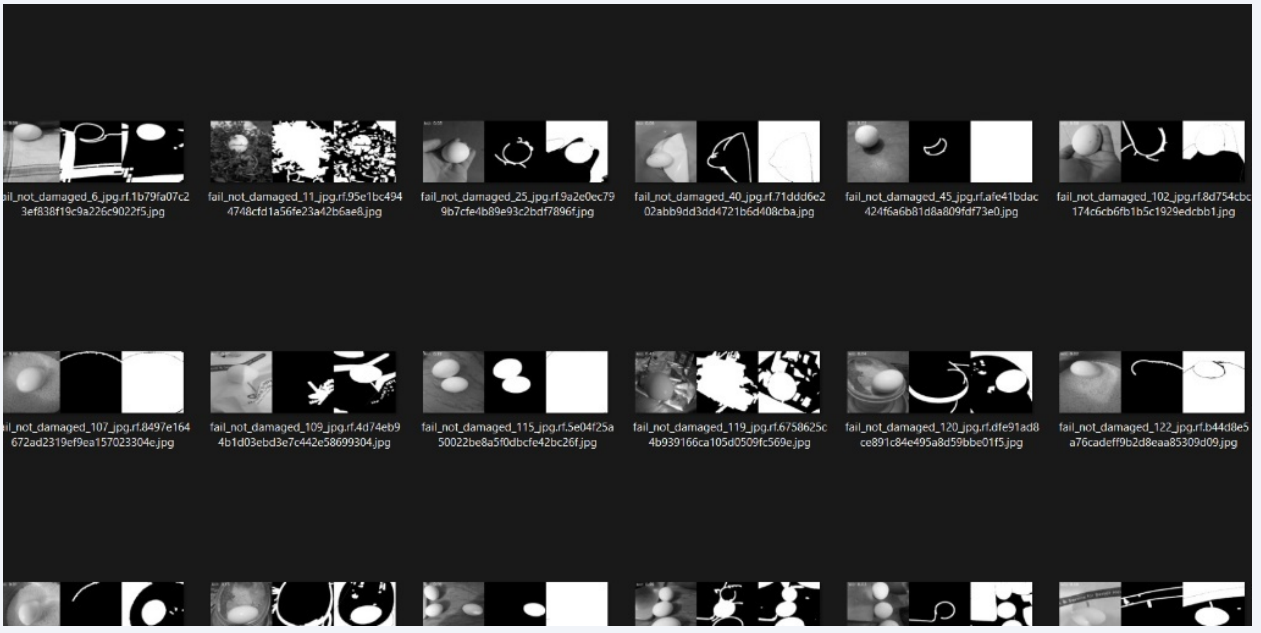


Figure 7: *Failure samples from bad_samples. Each group shows: grayscale input | classical Canny mask | unprompted SAM binary mask.*

Root cause analysis. Looking at Figures 6 and 7, two separate problems are pulling the score down, and I don't want to blame only one of them:

1. **The ground truth.** SAM was unprompted, so it segmented the hand, bowls and background as well as the egg. Once those polygons were merged into one binary mask, many ground-truth masks became mostly or fully white, which inflates the union and lowers IoU no matter how good the egg mask is.

2 **The classical pipeline.** Figure 7 also shows Canny failing on cluttered or low-contrast images. In some, the edge map stays as a thin open outline instead of a filled egg; in others, textured backgrounds produce large noisy blobs; and when two eggs touch, they merge into one region.

So 11.02% should not be read as the true accuracy of the pipeline, but it is not proof that the pipeline is fine either. The pipeline works well on clear, single-egg images and struggles on the rest, and a fairer score will need cleaner ground truth (see Section 7).

Evaluation log

Run	What changed	Average IoU	Finding
1	Local script, SAM overlays thresholded as masks	13.53%	SAM masks were solid white squares (RGB overlays, not binary).
2	Pure binary masks from `result.masks.xy`	11.02%	Unprompted SAM includes hand/background; Canny also fails on cluttered frames.

5. Task 4: Real-Time Video Tracking

For Task 4 I wrapped the Canny pipeline in a loop over time. My first plan was to compile the images into an MP4 with `cv2.VideoWriter`, but that failed because of Windows MSMF codec limitations: opening `egg_test_video.mp4` just reported "End of video stream reached". Instead of fighting the codec, I skipped the video file and fed the frame arrays straight into a playback loop, with `cv2.waitKey(500)` between frames (a 500 ms delay so the 22 frames don't flash past, and pressing **q** quits early). A full pass over the 22 frames takes about 11 seconds.

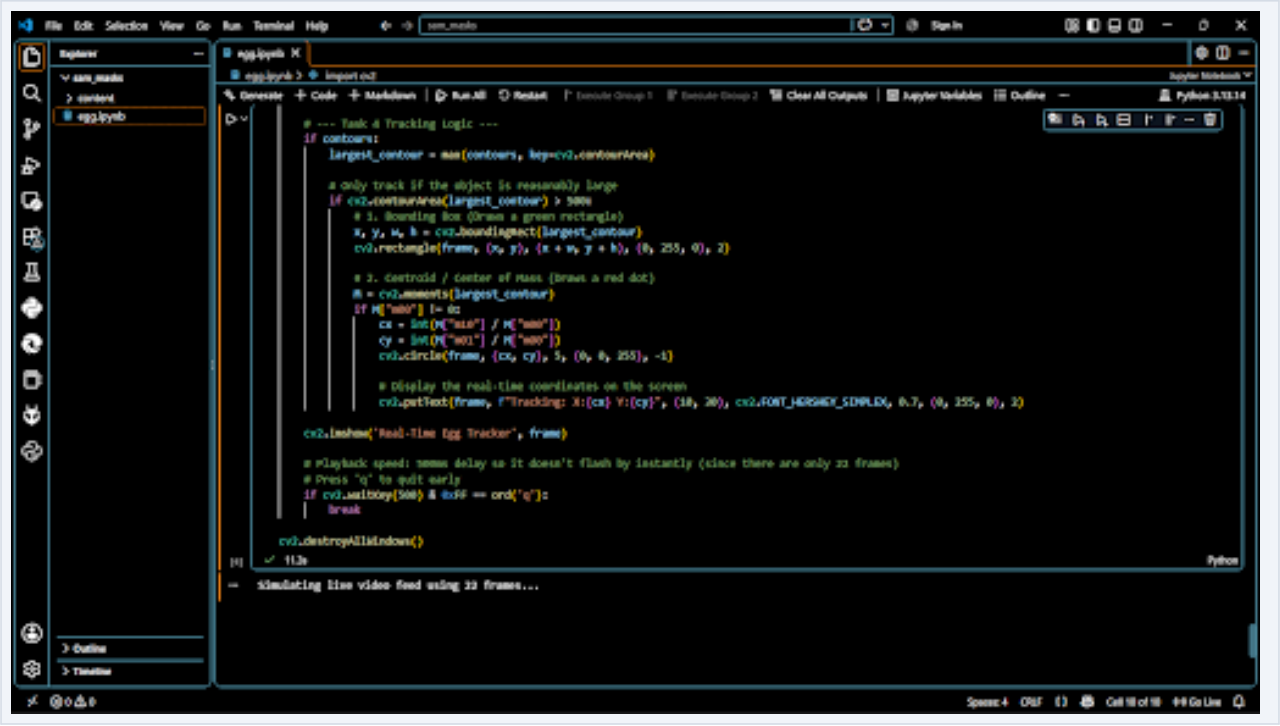


Figure 8: *VS Code running the real-time simulation loop over the validation frames.*

The tracking logic uses `cv2.boundingRect` to draw the green bounding box and `cv2.moments` to find the centroid (cx = m10 / m00, cy = m01 / m00), which is drawn as a red dot. The centroid coordinates are printed on the frame as "Tracking: X:… Y:…". A box is only drawn when the largest contour is big enough, to ignore small specks.



Figure 9: *Successful lock on a single egg (centroid X:259, Y:272).*



Figure 10: *Failure mode: several eggs and shadows merged into one bounding box (centroid X:224, Y:226).*

When there is one egg, as in Figure 9, the box fits the egg and the centroid sits on its centre. In Figure 10 the three touching eggs and their shadows become a single contour, so the box covers all of them and the centroid lands in between; the largest-contour rule can only ever return one object.

6. Limitations & Lessons Learned

- **One object per frame.** Keeping only the largest contour is what cleans the mask, but it also means touching eggs are merged and only one object can be reported (Figure 10).
- **Detection, not tracking.** Each frame is processed on its own. There are no object IDs and nothing links one frame to the next, so the "tracker" is really per-frame detection with a centroid.
- **Simulated video.** Because of the MP4 codec problem, tracking was demonstrated on a sequence of the 22 validation images rather than on a continuous video, so it was not tested against real motion blur or frame-to-frame movement.
- **Weak evaluation signal.** The 11.02% IoU is affected by both the unprompted SAM ground truth and the pipeline's own failures, so it is a rough indicator at best.
- **Sensitive parameters.** Canny thresholds, blur size and dilation settings are fixed by hand. They suit the images I looked at, but cluttered or low-contrast scenes need different values.
- **Lesson:** check the ground truth before trusting a metric. Two of my three "bad IoU" runs came from the evaluation setup, not the algorithm, and I only found that by looking at the saved images.

7. Future Recommendations & Scalability

Classical methods are fast and need no training, which suits controlled environments, but real-world use needs some upgrades:

- **Deep learning backbone:** replace Canny edge detection with an instance-segmentation network such as YOLOv8-seg, which copes with lighting changes and occlusion between objects.
- **Temporal filtering:** add a Kalman filter to the tracking loop to smooth centroid jitter and keep object IDs through short occlusions.
- **Cleaner ground truth:** prompt SAM with a box or point on the egg (or keep only the mask nearest the egg) so the reference mask contains only the egg, then re-score the classical pipeline.

- **Multi-egg handling:** process every sufficiently large contour instead of only the largest, and split touching eggs with a distance transform + watershed step so each egg gets its own box and centroid.
- **Real video test:** fix the codec problem (for example by writing frames with a different codec) and run the tracker on a continuous video clip.

8. Reproducibility

Key parameters of the final pipeline:

Stage	Setting
Pre-processing	Grayscale + Gaussian blur, (9, 9) kernel
Edge detection	Canny, thresholds 30 and 100
Morphology	Dilation, 5×5 kernel, 2 iterations
Contours	RETR_EXTERNAL; keep max contour by area; fill to make the mask
Evaluation	IoU vs. SAM binary masks on 22 validation images; images below 0.70 saved to bad_samples
Tracking display	cv2.boundingRect (green box), cv2.moments centroid (red dot), waitKey(500), q to quit

How to run it:

1. **Colab:** upload `egg.v1i.folder` (train and valid splits), install Ultralytics, and run SAM on the validation images; zip and download the predictions.
2. **Colab:** run the polygon-extraction loop on `result.masks.xy` to write pure binary masks, and download them as `pure_masks.zip`.
3. **Local (Windows):** unzip the masks next to the dataset, open `egg.ipynb` in VS Code (Python 3.13 with `opencv-python` and `numpy` installed), and run the Canny pipeline on the development images.
4. **Local:** run the IoU evaluation cell over the validation split; low-scoring images are written to `bad_samples`.
5. **Local:** run the tracking cell to see the live egg tracker over the 22 frames, and press **q** to stop.

9. AI Tools Used

SAM, accessed through the Ultralytics library, was used to generate the ground-truth masks, as described in Task 1. I also used an AI chat assistant during development, for example to get the idea for the trackbar GUI in the HSV trial and for help debugging path and evaluation-script errors. I ran all the code myself on Colab and on my local machine, and the experiments, results and failures described here come from those runs.

10. Conclusion

A classical pipeline built on Canny edges, dilation and largest-contour filtering gave a usable real-time egg locator for single-egg scenes, where fixed HSV thresholds and Otsu thresholding had both failed. The evaluation process taught me more than the algorithm did: the first two IoU numbers (13.53% and 11.02%) were dominated by problems in the ground truth, and the saved failure images also exposed real weaknesses in the classical pipeline. The next steps are cleaner ground truth, per-object handling for multiple eggs, and a learned segmentation model with a Kalman filter for stable tracking.